2025 24th International Symposium on Parallel and Distributed Computing (ISPDC) | 979-8-3315-7811-4/25/$31.00 ©2025 IEEE | DOI: 10.1109/ISPDC67428.2025.00027

Bitrate Hopping: An Intrusion Prevention Technique to Secure the CAN Bus-Based Distributed Network

William Luke Lambert
Tennessee Technological University

Sheikh Ghafoor
Tennessee Technological University

Samuel Hollifield
Oak Ridge National Laboratory

***Abstract*—Modern in-vehicle networks are increasingly vulnerable to cyberattacks due to the lack of built-in security in protocols such as the Controller Area Network (CAN). This paper introduces bitrate hopping, a proactive intrusion prevention technique that defends against message injection, denial-of-service (DoS), and passive eavesdropping. Bitrate hopping periodically changes the CAN bitrate among synchronized, trusted ECUs using a cryptographically secure pseudorandom sequence, preventing unauthorized nodes from communicating. Unlike cryptographic authentication mechanisms that often require a change in the CAN packet structure or add overhead that may result in violation of real-time constraints in some applications, bitrate hopping operates at the physical layer with minimal overhead. We first implement and evaluate a CAN-only version of bitrate hopping to maintain full backward compatibility with existing networks, demonstrating that it can be effective in short bursts to disrupt attacks without requiring additional hardware. Building on these results, we develop a more robust Ethernet-based implementation, which maintains consistently high message success rates at faster hop intervals and represents our primary focus for future system development. Our contributions include the design of a synchronization protocol for bitrate transitions, a dual-mode implementation over CAN and Ethernet, and an empirical study validating its feasibility and effectiveness.**

***Index Terms*—Bitrate hopping, Controller Area Network (CAN), Intrusion Prevention System (IPS), Distributed Systems, Cyber-Physical Systems (CPS), Real-time systems, Secure communication, Smart attacker, Embedded security, Automotive security**

I. Introduction

Modern automobiles face increased risks of cyberattacks that threaten safety and privacy. Attackers can exploit system vulnerabilities for unauthorized access, alteration of functions, or data extraction. Increased exposure through remote controls, over-the-air updates, and infotainment systems underscores the need for robust cybersecurity measures to maintain consumer trust. Cyberattacks now significantly affect vehicle operations and safety, especially by targeting cloud infrastructure and internal networks. Cloud-based attacks can extract private information and allow programmatic interactions through vulnerable application programming interfaces. Attacks on in-vehicle networks directly impact a vehicle's internal systems. Modern vehicles use a network of Electronic Control Units (ECUs) for crucial functions, with the Controller Area Network (CAN) being the most common due to its cost-effectiveness and efficiency. Despite its practicality, CAN lacks inherent security and can be exploited to control functions such as braking or steering, increasing the risk of accidents [3], [10], [20]. A notable example was the Jeep Hack by Miller and Valasek [12], which demonstrated remote hijacking by reprogramming a link to the CAN bus. As vehicles evolve toward autonomy, systems that contain technologies such as Wi-Fi, Bluetooth, and more become vulnerable, expanding the attack surface. Research into CAN bus security has focused on Intrusion Detection Systems (IDS) and Attack Prevention Techniques. IDS can be Signature-Based, Anomaly-Based, or Hybrid, while prevention techniques include Message Authentication Codes (MACs) for message integrity and access controls for authorization. Although attack detection is a growing area of research, preventive techniques are the focus of this publication.

In many real-world situations, shutting down or immobilizing a vehicle due to a misbehaving ECU may not be a safe or acceptable option, especially if the vehicle is in motion or located in a critical environment such as a highway or construction zone. In addition, stopping a vehicle may be the ultimate objective of the attacker. Bitrate hopping offers a proactive security mechanism that enables the system to enter a degraded but secure operational state, similar to a 'limp-home' mode. This approach isolates the attack surface from the rest of the vehicle network, preventing further spread of malicious activity while allowing core safety functions to continue to operate. Importantly, bitrate hopping does not assume that the attacker is the ECU itself; rather, it accounts for scenarios where legitimate but vulnerable ECUs, such as infotainment systems, have been indirectly compromised by remote exploitation or physical access.

In this paper, we make the following contributions.

- We propose a novel physical layer intrusion prevention mechanism, bitrate hopping, that prevents adversaries from synchronizing with the CAN bus by altering bitrates in a pseudorandom, forward secret sequence.
- We design two implementations of bitrate hopping: one using Ethernet for coordination and another using a single CAN channel to maintain backward compatibility.
- We implement and test the system on Raspberry Pis and show that even at high hop frequencies (e.g. 10 to 1000 ms), the message success rate remains high while attackers are desynchronized.
- We evaluated resilience to message injection and eavesdropping attacks, showing that bitrate hopping forces attackers into error states or obscures message interpretation.

Initially, we developed bitrate hopping as a CAN-only system to ensure backward compatibility with traditional CAN

979-8-3315-7811-4/25/$31.00 ©2025 IEEE
DOI 10.1109/ISPDC67428.2025.00027

Authorized licensed use limited to: Notre Dame University - Louaize. Downloaded on September 21,2026 at 19:58:19 UTC from IEEE Xplore. Restrictions apply.

2.0 networks. Although this approach does not require additional hardware and offers a lightweight, temporary defense against active attackers, its reliability is constrained at very fast hop intervals. Recognizing these limitations, we extended the design to support Ethernet-based coordination between nodes. This Ethernet-capable version demonstrates superior synchronization precision and message success rates, making it the primary focus of our ongoing and future work. However, the CAN-only version remains a viable option for rapid deployment in constrained environments.

II. Background

A. Overview of the CAN Bus Protocol

The CAN bus is a two-wire half-duplex high-speed serial network technology developed by Robert Bosch GmbH. It is internationally standardized according to ISO 11898 [9]. As a broadcast protocol, it links multiple low-power simple ECUs in modern vehicles.

1) CAN Vulnerabilities: The CAN protocol lacks integrated security features such as authentication, encryption, and prevention of denial of service (DoS) attacks. These omissions leave the CAN vulnerable to exploitation. Additionally, CAN's priority-based arbitration is intrinsically vulnerable. We have identified seven categories of CAN vulnerabilities: *absence of source or destination addresses*, *missing built-in clock or sequence number*, *priority-based arbitration*, *no encryption mechanism*, *flawed error handling*, *lack of protection against malicious firmware updates*, and *absence of mechanisms for detecting unauthorized devices on the CAN bus* [19].

B. Intrusion Prevention Systems

An intrusion prevention system (IPS) in cyber security for the controller area network (CAN) is a proactive tool designed to detect and block unauthorized access or malicious actions within a CAN. Techniques such as signature-based detection, anomaly detection, and behavior analysis are used to detect threats. Upon detecting a threat, the IPS promptly takes action to counter or minimize the attack, safeguarding the CAN network from further harm. This proactive security ensures the integrity, confidentiality, and availability of the CAN network, supporting the seamless operation of interconnected devices.

C. Frequency Hopping

Frequency hopping is a technique used in wireless communication systems to improve transmission security and reliability. It involves rapidly changing the frequency at which the signal is transmitted, jumping between different frequencies in a predetermined pattern. This helps minimize interference from other devices and makes it more difficult for eavesdroppers to intercept the signal. Frequency hopping is commonly used in technologies such as Bluetooth and spread-spectrum systems to prevent eavesdropping and other attacks.

III. Related Work

A. Attack Prevention Techniques

Current CAN attack prevention research can be broadly separated into two main categories: using packet- or message-based authentication (e.g., Message Authentication Codes (MACs)) and side-channel implementations of existing CAN protocol technologies (such as CAN error handling for invalid authentications). Despite various Message Authentication Code (MAC) techniques to thwart masquerade attacks, they do not prevent replay attacks without counters due to CAN bus timing limits. Nilsson et al. [14] and Szilagyi et al. [16] suggested MAC use but faced communication delays and size issues; Schweppe et al. [15] advocated hardware modules, but ignored large payload handling; and Lin et al. [10] employed counters while leaving DoS attacks unaddressed. Other MAC-based solutions, such as CANAuth by Van Herrewege et al. [18] and SecCAN by Ullah et al. [17] raised compatibility and resource concerns. Defending against unauthorized CAN messages, Matsumoto et al.'s IPS [11] enforced bus-off by error frames but struggled with backward compatibility and susceptibility to DoS attacks. Dagan et al.'s Parrot [4] increased attackers' error counters leading them to a bus-off state but hampered real-time processing. This is an example of a bus-off attack in which an attacker uses CAN error frames to disable specific ECUs on the CAN bus. Similarly, the error frame IPS by De et al. [5] effectively discarded malicious frames, however, like Parrot, is limited by real-time processing. Additional strategies like RC4 encryption by Chavez et al. [1] and CANeleon by Cheng et al. [2] bring security improvements but face cost or priority DoS issues.

B. Research Gaps

The aforementioned research works use various techniques to enhance security in CAN bus networks but do not effectively prevent DoS or bus-off attacks. For example, these prevention techniques do not prevent a DoS attack where messages with ID 0x000 (the highest priority possible in CAN) is sent flooded to the vehicle's CAN bus, a basic attack that even novices can execute. The bus-off attack is not adequately addressed, with only one work [6] that mitigates it by verifying ECU certificates. Most authentication mechanisms require altering the CAN frame for the MAC, which conflicts with CAN 2.0's backward compatibility. Of the articles reviewed, many did not test their proposals in real time. Key research gaps include overemphasis on the OBD-II port over other vehicle attack surfaces, a lack of DoS prevention strategies that do not modify the transceiver or counterattack ECUs, limited solutions to forestall bus-off attacks post-masquerade, and prevention methods based on message authentication codes, which do not fit into standard CAN 2.0 frames or suggest substantial modifications to CAN's architecture.

C. Comparison with Existing Prevention Techniques

Compared to Message Authentication Code (MAC)-based solutions like CANAuth [7] or HMAC-based models [14], bitrate hopping offers a lightweight alternative that avoids

Authorized licensed use limited to: Notre Dame University - Louaize. Downloaded on September 21,2026 at 19:58:19 UTC from IEEE Xplore. Restrictions apply.

modifying the CAN frame structure or incurring per-message cryptographic costs. Error-handling defenses, such as Parrot [4], can isolate attackers after detection, but do not prevent message injection or eavesdropping in real time. Bitrate hopping, by contrast, is preventive in nature and applies uniformly regardless of attack visibility. Although prior work like Matsumoto et al. [11] attempts proactive suppression, it still relies on standard bitrates and lacks temporal unpredictability.

D. Objectives

To address these research gaps, we introduce bitrate hopping for intrusion prevention. This technique is designed to thwart both DoS and passive eavesdropping by taking cues from frequency hopping, which is commonly used to mitigate radio interference and interception. Bitrate hopping involves swiftly altering the bitrate of attack surfaces and linked ECUs. This strategy seeks to interrupt an attacker's ability to consistently engage with the CAN bus. Upon detection of an attack, the bitrates of the IPS and associated ECUs will change periodically and in sync. To prevent the attacker from adapting to this approach, a pseudorandom algorithm with forward secrecy governs the timing and selection of bitrate hops.

By cutting off the attacker's communication, this method hinders the continuation of DoS attacks within the CAN bus, since the attacker must discern the correct bitrate for message transmission. Similarly, it can prevent passive eavesdropping. Additionally, rather than halting communication with the ECUs exposed to the attack surface, this method allows messages to be continuously passed while the bitrates fluctuate. The primary aim of this research is to create an IPS positioned between potential attack points and the ECUs connected to them. This system will leverage bitrate hopping to disrupt communications from potential attackers while maintaining seamless communication with legitimate messages originating from attack surfaces. The proposed methodology aims to effectively counteract both DoS and eavesdropping attacks through a pseudo-random hopping sequence that effectively obstructs attack prediction.

IV. System and Threat Model

A. System Overview

The proposed intrusion prevention system (IPS) uses bitrate hopping to prevent CAN-based attacks by making it difficult for unauthorized nodes to maintain synchronized communication. The IPS may be implemented as an in-line gateway device between the vehicle's attack surface (e.g., OBD-II port, telematics unit) and one or more Electronic Control Units (ECUs), or distributed across ECUs with pre-agreed coordination. The system consists of a master node (e.g. a hardened ECU or the IPS device) that generates a pseudorandom sequence of bitrate changes using a cryptographically secure pseudorandom function (CSPRNG). At a fixed hop frequency, this master node synchronizes the next bitrate with authorized ECUs via either of the following coordination methods.

- Single-channel CAN coordination: The bitrate changes are embedded in encrypted CAN messages using XOR and ChaCha20. Soft resynchronization mechanisms are used to recover from missed synchronization.
- Ethernet-based synchronization: A TCP-based control channel is used to securely communicate the next bitrate. This method assumes the availability of an out-of-band automotive Ethernet interface.

Each participating ECU transitions to the next bitrate according to this schedule. Nodes that are not synchronized will encounter repeated CAN errors, causing them to fall into an error-active or bus-off state.

The IPS is not intended to sanitize or control already-compromised ECUs (e.g., a hijacked infotainment system). Instead, it acts as a boundary enforcement mechanism, isolating high-risk entry points, such as telematics units, OBD-II ports, or infotainment systems, from safety-critical components. Once an attack is detected or suspected, the IPS shifts communication with trusted ECUs to coordinated hopping, which the attacker cannot follow unless fully synchronized. This prevents lateral movement within the CAN bus, even if the attack originates from a device that the IPS cannot directly manage.

B. Threat Model

The adversary is assumed to have on-bus access to the CAN network (e.g., through the OBD-II port or reverse-engineered telematics ECU) or via remote compromise. This threat model aligns with real-world exploits such as the Jeep Cherokee hack [13].

Attacker capabilities include:

- Eavesdropping on unencrypted CAN traffic to extract message identifiers and timing.
- Injecting arbitrary messages using a modified CAN transceiver.
- Launching DoS attacks by exploiting the arbitration mechanism with dominant bits or high-priority flooding.
- Attempting to guess the current bitrate in real time through brute-force attempts.

However, the attacker is constrained in the following ways:

- They cannot decrypt bitrate coordination messages or predict future bitrate changes due to the forward secrecy of the pseudorandom function.
- They do not have access to the preshared keys or initial nonce used in the secure key exchange.

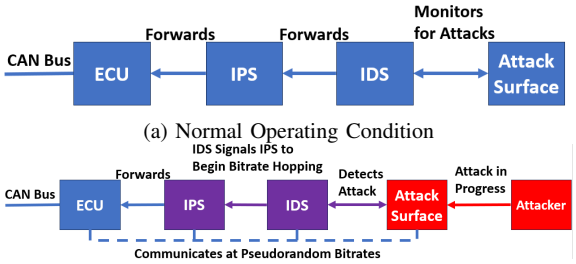
In addition to conventional adversaries operating at a fixed bitrate or trying to inject naively, we also consider more capable threats. A stronger adversary may attempt partial synchronization by analyzing timing patterns, launching adaptive brute-force bitrate cycling, or inferring bitrates through error feedback. Although the current design already limits the success of such attacks through rapid bitrate changes and encrypted coordination, future iterations will incorporate adaptive hopping strategies and timing obfuscation to further mitigate these risks. A prototype of a 'smart attacker', designed

Authorized licensed use limited to: Notre Dame University - Louaize. Downloaded on September 21,2026 at 19:58:19 UTC from IEEE Xplore. Restrictions apply.

to test the limits of such inference-based strategies, is in active development and will be used to adjust system parameters and assess resilience under adversarial learning conditions.

This model represents a strong but realistic attacker with access to the CAN bus but no ability to break cryptographic primitives or compromise secure ECUs.

V. System Architecture



(a) Normal Operating Condition

(b) Under Attack

Fig. 1: Architecture

Our Intrusion Prevention System (IPS), seen in Figure 1. When the IDS detects an attack, it informs the IPS. At point the IPS initiate the bit rate hopping algorithm to prevent the attack. The focus of this paper is restricted to attack prevention rather than the specifics of detection techniques. During normal operation, as seen in Figure 1a, the IDS component of the IPS monitors attacks, while the IPS forwards messages from the attack surfaces to the connected Electronic Control Unit (ECU). Once an attack is detected, as seen in Figure 1b, both the IPS and the connected ECU initiate periodic changes in their bitrates in a synchronized manner. Instead of completely severing communication with the affected attack surface, the IPS continues to send messages while rapidly altering bitrates. This effectively enables the IPS to function in a ”limp-home” mode. To prevent attackers from predicting and countering bitrate changes, a pseudorandom algorithm is utilized. This algorithm determines the specific bitrates to which the devices are set, as well as the duration for which they will remain at each bitrate.

VI. Prototype Implementation

We developed a prototype for bitrate hopping utilizing two interconnected Raspberry Pi 4 units, connected via CAN and Ethernet, and programmed them using Python 3. To maintain synchronized timing between the two Raspberry Pis, one device operates as a local NTP server to align the time with the other Raspberry Pi, which functions as an NTP client. The system operates by having one Raspberry Pi (RPi1) periodically sends an encrypted ”heartbeat” message that specifies the next bitrate to which all nodes must switch. RPi1 then switches to this bitrate. The second Raspberry Pi (RPi2) receives the encrypted heartbeat, decrypts it, and then adjusts to the specified bitrate to maintain synchronization with RPi1.

We developed a simple experiment to evaluate the practicality of employing unconventional bitrates for communication. In the automotive sector, vehicles typically operate within bitrate ranges of 500 to 150 kb / s, and the usage of atypical bitrates like 123,123 b/s is rare. we executed the initial test on both a Raspberry Pi 3 and a Raspberry Pi 4, each fitted with a 2-Channel Isolated CAN FD Expansion HAT. This equipment facilitates communication through the CAN 2.0 and CAN FD protocols using SocketCAN, which comprises a suite of open-source CAN drivers and a customized networking stack for Linux systems. Using the can0 and can1 channels, linked through CAN-H and CAN-L, we verified the feasibility of unconventional bitrates. Furthermore, the experiment demonstrated that the two CAN channels can communicate effectively with differing bitrates. For example, can0 operating at 500 Kb/s can communicate successfully with can1 at 496.914 Kb/s. Following these results, we conducted a more thorough investigation to determine the maximum and minimum communication distances for seven different bitrates, detailed in Table I: 3 Mb/s, 1 Mb/s, 500 Kb/s, 250 Kb/s, 125 Kb/s, 40 Kb/s, and 5 Kb/s.

TABLE I: Raspberry Pi Bitrate Testing

can0 Bitrate	can1 Bitrate Range	Lower Bound	Upper Bound
3000000 (3 Mb/s)	2967033–3205127	32968 (1.10%)	205128 (6.84%)
1000000 (1 Mb/s)	987804–1012820	12196 (1.22%)	12821 (1.28%)
500000 (500 Kb/s)	496914–503164	3087 (0.62%)	3164 (0.63%)
250000 (250 Kb/s)	249224–250786	777 (0.31%)	787 (0.31%)
125000 (125 Kb/s)	123648–125588	1353 (1.08%)	589 (0.47%)
40000 (40 Kb/s)	39861–40099	140 (0.35%)	100 (0.25%)
5000 (5 Kb/s)	4980–5025	20 (0.40%)	25 (0.50%)

The system design evolved through two primary implementations: first, a CAN-only version that relies solely on the CAN bus for synchronization, and second, an Ethernet-based version that utilizes TCP communication for improved timing precision.

Initially, bitrate hopping was designed to operate using only CAN communication to maintain full backward compatibility with legacy vehicles. Although the CAN-only method offers the advantage of not requiring additional hardware interfaces, it faces limitations at very fast hop frequencies, impacting message success rates. However, it remains a practical lightweight option for rapid, temporary disruption of attacks without retrofitting vehicle networks.

Subsequently, we extended our design to utilize an Ethernet coordination channel. Although this requires vehicles to support an Ethernet-capable interface between the ECUs, it does not require specialized automotive Ethernet standards. Any standard Ethernet link that can support TCP communication between nodes is sufficient. This extension improves synchronization precision and maintains very high message reliability across a wide range of hop frequencies. The Ethernet-based model is the primary direction for future system development due to its robustness, but the CAN-only model remains a viable fallback for constrained environments.

After a secure key exchange, the bitrates are encrypted and decrypted using the ChaCha20 encryption protocol, a 256-

Authorized licensed use limited to: Notre Dame University - Louaize. Downloaded on September 21,2026 at 19:58:19 UTC from IEEE Xplore. Restrictions apply.

bit stream cipher. ChaCha20 was specifically chosen for its efficiency in encrypting small data payloads, such as the 8-byte maximum size of a Controller Area Network (CAN) frame, while maintaining robust security. New bitrates are generated using a novel pseudo-random function, which uses Python's secrets library to ensure forward secrecy.

A. CAN-Only Bitrate Hopping Implementation

In this approach, one Raspberry Pi (RPi1) acts as a server, creating and distributing new bitrates, while the second Raspberry Pi (RPi2) operates as a client, listening for and switching to new bitrates. Because a standard CAN frame cannot accommodate a full 32-byte key exchange payload, this method uses XOR encryption with an 8-byte key and nonce. The nonce combines a random seed and an incremental counter for enhanced unpredictability. Although less secure than ECC-based methods, this design choice balances security and message size constraints. ChaCha20 is used to encrypt heartbeat messages that convey the next bitrate. Soft resynchronization mechanisms are implemented: if a client detects a missing message beyond a threshold, it reverts to a predefined bitrate and awaits server-driven resynchronization.

To handle temporary desynchronization, client nodes revert to a fallback bitrate and wait for the next synchronization message from the controller. This allows rejoining the hopping sequence without full reinitialization, reducing recovery latency.

Although the CAN-only system is limited by synchronization noise and timing precision compared to Ethernet, it remains fully backward compatible with existing CAN 2.0 and J1939 implementations and provides a practical short-term defense mechanism without requiring network upgrades.

B. Ethernet-Coordinated Bitrate Hopping Implementation

In this version, RPi1 operates as a TCP server and RPi2 as a client, communicating over a standard Ethernet link. Upon initial connection, the two devices perform an Elliptic Curve Cryptography (ECC)-based key exchange to establish a secure communication channel. ECC was selected because of its efficiency on resource-constrained devices, which provides strong security with a minimal performance penalty. ChaCha20 encrypts all follow-up communications post-key exchange. This method requires only that ECUs or gateways support Ethernet communication between nodes and does not depend on specialized automotive Ethernet standards. After establishing the secure channel, the server periodically sends encrypted TCP packets containing the next bitrate, allowing highly precise synchronization across a wide range of hop frequencies.

C. Bitrate Hopping Algorithm Description

The bitrate hopping algorithm dynamically adjusts the CAN bus bitrate in a synchronized yet unpredictable manner to disrupt attackers while maintaining communication among legitimate nodes. It leverages cryptographically secure randomness while bounding adjustments within predefined safe operational limits.

The system first defines a minimum bitrate (B_{min}) and a maximum bitrate (B_{max}). For example:

$$B_{\text{min}} = 500{,}000 \times \left(1 + \frac{1.5 \times 3}{100}\right) = 522{,}500 \text{ bps}$$

$$B_{\text{max}} = 600{,}000 \text{ bps}$$

The algorithm executes the following loop:

- Retrieve the current bitrate B.
- Randomly select a multiplier $m \in \{1, 2, 3\}$ and a direction $d \in \{-1, +1\}$ (decrease or increase).
- Compute the new bitrate candidate:

$$B' = B + \text{round}\left(B \times \frac{1.5 \times m}{100} \times d\right)$$

- Clamp B' within [B_{min}, B_{max}].
- If $B' \neq B$, broadcast the new bitrate and apply it locally.
- Sleep for the configured hopping interval and repeat.

Algorithm 1 Bitrate Hopping Algorithm

```
Define B_min and B_max
while true do
    B ← current bitrate
    repeat
        Randomly select m ∈ {1,2,3} and d ∈ {−1,+1}
        B' ← round adjusted bitrate
        Clamp B' within [B_min, B_max]
    until B' ≠ B
    Broadcast B', update local bitrate
    Sleep for hopping interval
end while
```

This bounded, pseudorandom alteration complicates attacker synchronization without introducing excessive disruption. The hopping parameters: base percentage, multiplier range, and hop frequency should be tuned based on the criticality of the protected system. Safety-critical systems may prefer conservative hopping, while non-critical systems may tolerate faster and more aggressive changes. Future work aims to further optimize the hopping strategy for robustness against advanced attackers, including those capable of partial bitrate inference or brute-force resynchronization.

D. Experimental Setup and Measurement Procedure

A detailed experiment was conducted in which one hundred thousand to one million messages were sent from one raspberry pi (sender) to another raspberry pi (receiver) in different configurations, using both the Ethernet and CAN-only versions. Specifically, one million messages were sent at a frequency of 10 ms and one hundred thousand at 100 ms, both common frequencies seen in CAN and J1939. These values were chosen to restrict all experiments to approximately 2.8 hours of simulated traffic generation. The following bitrate hop rates were tested: 0 ms (no hopping), 0.001 ms, 0.005 ms, 0.01 ms, 0.05 ms, 0.1 ms, 0.5 ms, 1 ms, 10 ms, 100 ms, 500

Authorized licensed use limited to: Notre Dame University - Louaize. Downloaded on September 21,2026 at 19:58:19 UTC from IEEE Xplore. Restrictions apply.

ms, 1000 ms, 1500 ms, and 2000 ms. Each message sent by the sender consists of the same arbitration ID, 0x100, and the current timestamp. When the receiver receives a message, it extracts the timestamp and then subtracts it from its new current timestamp to calculate the delay for each message. When sending stops, the receiver creates a csv file containing the sender's timestamp, the receiver's timestamp, the added delay, the average delay, and the total number of messages received. Because a CAN error occurs when a message is sent or received while a bitrate hop is in progress, the sender is programmed to immediately increment its counter and skip to the next message rather than continually retrying to send the message until successful. Similarly, when the receiver detects an error, it immediately resumes listening for messages.

E. Performance Results and System Behavior

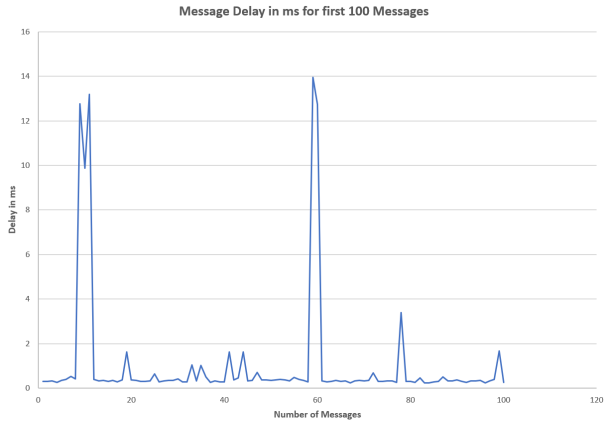


Fig. 2: Scatter plot of message transmission delays during bitrate hopping.

Figure 2 illustrates a scatter plot of message transmission delays during a representative bitrate hopping experiment. In this test, messages were sent at a rate of 100 ms while bitrate hopping occurred at a 1 ms interval. Although most messages exhibit minimal added latency, occasional delay spikes can be observed, corresponding to synchronization transitions or transient desynchronization between nodes. Despite these spikes, the average communication delay across all messages remains consistently under 1 millisecond, satisfying the hard real-time requirements of CAN systems.

Importantly, this minor self-healing disruption represents an intentional trade-off to prevent far more serious attacks. Without proactive defense like bitrate hopping, Denial-of-Service (DoS) and bus-off attacks could fully disable critical vehicle systems by overwhelming the CAN bus or forcing ECUs offline. Bitrate hopping thwarts such attacks by immediately severing the attacker's ability to maintain communication, preserving system availability and safety. Thus, a slight impact on the delay and delivery of the message is a necessary and justified compromise to maintain continuous operational security in the presence of active threats.

TABLE II: Bitrate Hopping Results Using CAN-Only Communication

Hop Frequency (ms)	10 ms Traffic		100 ms Traffic	
	Success %	Avg Latency / Hop (ms)	Success %	Avg Latency / Hop (ms)
0	100.00	0.0000	100.00	0.0000
0.001	44.99	5.8489	45.10	6.1293
0.005	44.99	5.8802	45.50	5.5813
0.01	45.23	5.3328	45.06	5.5158
0.05	44.67	5.5682	46.13	4.6608
0.1	44.80	5.6400	46.13	4.6321
0.5	72.20	1.5452	46.13	4.5927
1.0	72.20	1.5250	46.13	4.5540
10	72.20	1.5559	46.13	4.6035
100	82.99	1.7782	87.74	1.5222
500	62.99	2.4296	63.79	3.3042
1000	48.80	1.9726	63.79	3.2976
1500	43.10	1.9063	54.10	3.5408
2000	42.02	2.0223	47.60	3.6191

1) Evaluation: CAN-Only Transmission Performance: Table II illustrates the performance of bitrate hopping using only a single CAN channel at various hop frequencies for traffic scenarios of 10 and 100 ms.

In the 10 ms setup, message success rates are significantly lower at very fast hop intervals (e.g., around 45% success at 0.001–0.01 ms), but improve significantly as the hop frequency slows towards 0.5 ms and 1 ms (approximately 72% success). Moderate hop frequencies achieve a balance between maintaining availability and minimizing added latency (around 1.5 ms).

In the 100 ms traffic setup, slower hop frequencies ($\geq$100 ms) yield the best results, with success rates reaching 87.74% and relatively low added latencies (1.5 ms).

These findings suggest that CAN-only bitrate hopping is best suited for slower hop rates or short-term deployment during active attack periods. Although less reliable than Ethernet coordination at fast hopping intervals, it maintains full backward compatibility with standard CAN 2.0 systems and requires no additional hardware, making it a practical lightweight countermeasure in resource-constrained scenarios.

TABLE III: Bitrate Hopping Results Using Ethernet-Based Communication

Hop Frequency (ms)	10 ms Traffic		100 ms Traffic	
	Success %	Avg Latency / Hop (ms)	Success %	Avg Latency / Hop (ms)
0	100.00	0.0000	100.00	0.0000
0.001	95.26	1.0583	98.03	0.8053
0.005	95.10	0.7598	97.08	0.6418
0.01	95.13	0.8068	95.87	0.5815
0.05	95.10	0.9139	98.55	0.6594
0.1	95.21	0.8003	97.61	0.5752
0.5	95.57	0.8409	99.01	0.7721
1.0	96.82	0.7712	98.87	0.6059
10	96.83	0.6543	99.10	0.5122
100	98.48	0.7377	99.37	0.7163
500	99.52	0.6846	99.52	0.6640
1000	99.83	0.7392	99.81	0.5183
1500	99.90	0.5842	99.91	0.2517
2000	99.91	0.4189	99.91	0.2307

2) Evaluation: Ethernet-Based Transmission Performance: Table III illustrates the performance of bitrate hopping when using Ethernet-based coordination.

In this configuration, even fast hopping intervals (for example, 1 ms or 10 ms) maintain message success rates above 95%, with extremely low added latency ($<$1 ms in most cases). At slower hop frequencies ($\geq$100 ms), success rates consistently exceed 99%, and added latency becomes negligible.

Authorized licensed use limited to: Notre Dame University - Louaize. Downloaded on September 21,2026 at 19:58:19 UTC from IEEE Xplore. Restrictions apply.

This shows that Ethernet-coordinated bitrate hopping achieves significantly higher reliability and timing precision compared to the CAN-only model. Although it requires ECUs or gateways equipped with Ethernet communication capability, it offers a superior long-term solution to protect in-vehicle networks against persistent attacks and does not require any modification to the CAN frames.

3) Security Evaluation Under Attack Scenarios: To test the effectiveness of bitrate hopping against attacks, two initial attack scenarios were conceived: denial of service through message injection of dominant bits and eavesdropping of CAN message contents. These attacks were performed using a Kvaser Leaf Light v2 connected to a Windows 11 laptop connected to our two Raspberry Pis. To perform a DoS, high-priority messages using an arbitration ID of 0x000 and a payload of 0x00 were sent rapidly into the bus at a rate of 0.001 ms. Eavesdropping was performed by simply viewing the CAN dump during communication between the two raspberry pis. In both CAN-only and Ethernet-coordinated setups, when bitrate hopping occurs during the denial of service attack, the attacking node receives error frames and is forced into an error-active state so that it cannot continue to send or receive CAN traffic from the victim, ensuring secure and continuous communication between legitimate nodes and effectively isolating attackers from their victims. During attempted eavesdropping, the CAN messages were malformed or the attacking device was overloaded with error frames instead of legitimate messages. These findings indicate that bitrate hopping could serve as an effective preventive method within the frameworks of CAN and J1939. Similarly to other countermeasures, this technique effectively mitigates 0-based DoS attacks. However, an emphasis is also placed on preventing unauthenticated reading of CAN messages, which is not fully considered in current attack prevention methodologies.

Bitrate hopping offers a fundamentally different defense mechanism compared to existing CAN-layer intrusion prevention approaches. Bus-off-based countermeasures, such as Parrot [4] and error-based suppression schemes like those proposed by Matsumoto et al. [11], work by inducing fault states in attackers after detection. However, these methods react only after malicious activity has occurred, and often introduce downtime or risk false positives. In contrast, bitrate hopping prevents unauthorized nodes from maintaining synchronization, thereby proactively disrupting message injection and passive sniffing before they succeed. This ensures continuous communication among trusted nodes without requiring post-attack recovery. Furthermore, while authentication-based defenses (e.g., CANAuth [8]) focus on verifying the origin of the message, they do not prevent attackers from transmitting valid looking frames. Bitrate hopping disrupts message delivery itself by exploiting attackers' inability to track dynamic bitrate changes, making it especially effective against time-based or fuzzing-style DoS attacks.

VII. Discussion and Interpretation

These results show that bitrate hopping is effective in disrupting attacker synchronization while maintaining relatively high message success rates across a wide range of hop frequencies. However, a deeper interpretation is required to understand the implications of these findings for real-world in-vehicle communication.

A. Message Success Tradeoffs

Message success rates are closely tied to hop frequency and coordination method. In the Ethernet-based system, success rates remain above 95% even at fast intervals (e.g. 1 ms). In contrast, the CAN-only method suffers substantial reliability loss at high hopping speeds (e.g. 45% success at 1 ms), due to the half-duplex nature of CAN and the absence of an out-of-band channel. Both methods perform well at slower hop rates ($\geq$100 ms), but the CAN-only approach imposes a steeper trade-off between security and reliability, making it better suited for short-term isolation rather than sustained communication during attacks.

B. Latency and Safety-Critical Impact

The average added latency remains below 1 millisecond in most hop frequencies, which is acceptable for real-time systems. However, small delays can still affect safety-critical functions like braking and steering. Because the current system does not implement automatic retransmission, the messages dropped are permanently lost. Future iterations should explore retransmission queues or leverage CAN FD redundancy.

Although high-frequency hopping does reduce message success rate in some cases, most configurations of the Ethernet-based approach maintain sub-millisecond latency, which is within real-time CAN limits. For safety-critical ECUs, such as braking or steering modules, the system allows slower hop intervals or even static bitrate assignment to preserve deterministic behavior. Furthermore, CAN FD or redundant channels can be used to ensure reliability. Finally, we are investigating future versions of bitrate hopping which may incorporate retransmission queues or dedicated, non-hopping CAN channels for certain critical messages.

C. System Responsiveness and Attack Recovery

In both versions, attackers are quickly desynchronized and forced into error-active states without needing message filtering. Ethernet-based coordination exhibits faster and more robust resynchronization due to precise out-of-band timing, while CAN-only synchronization, although improved with soft resynchronization mechanisms, remains vulnerable to transient gaps. This suggests that Ethernet is the preferable long-term solution, while CAN-only provides a lightweight short-term defense.

D. Deployment Considerations

The deployment of bitrate hopping in production vehicles requires careful integration. ECU firmware must support dynamic bitrate switching, or a gateway IPS must bridge legacy

Authorized licensed use limited to: Notre Dame University - Louaize. Downloaded on September 21,2026 at 19:58:19 UTC from IEEE Xplore. Restrictions apply.

systems. Hop frequencies must balance disruption effectiveness against communication stability.

Scaling bitrate hopping to real automotive ECUs introduces timing and jitter constraints that must be evaluated under ISO 11898 conditions. Our current prototype uses Raspberry Pi nodes for development purposes, but future work will involve testing on automotive-grade microcontrollers and transceivers using Hardware-in-the-Loop (HIL) and real ECU deployments.

E. Comparison with Other Intrusion Prevention Techniques

Unlike reactive systems such as bus off enforcement [4], [11], which depend on attack detection and can be abused, bitrate hopping proactively prevents unauthorized communication by disrupting physical-layer synchronization. Compared to MAC authentication protocols [14], [18], bitrate hopping avoids modifying CAN frames or introducing cryptographic latency, preserving compatibility with CAN 2.0.

F. Limitations

Although bitrate hopping offers a proactive defense, there are several limitations. It requires precise timing, particularly in CAN-only systems, where desynchronization can lead to legitimate message loss. At high hopping frequencies ($\leq$10 ms), message success degrades, which can impact safety-critical messaging. The deployment of bitrate hopping requires modified ECUs or gateways, and the system assumes that attackers cannot compromise hopping schedules.

Current evaluations are limited to Raspberry Pi testbeds, and real ECU testing is needed to validate scalability. Finally, while delay spikes during hopping are minor, average delays remain under 1 ms, and their specific variability characteristics are under investigation and will be addressed in future work.

VIII. Future Work

Although bitrate hopping shows strong potential as a proactive intrusion prevention mechanism, several avenues for future research remain open to improve scalability, resilience, and integration with production automotive systems.

A. Validation in Production-Grade Environments

The current prototype was evaluated using Raspberry Pi-based nodes in a controlled laboratory environment. Future work will involve deployment on production-grade automotive microcontrollers and CAN transceivers that conform to the ISO 11898 specifications. We intend to conduct hardware-in-the-loop (HIL) testing using commercial CAN development kits to evaluate timing behavior, electrical tolerances, and error recovery under more realistic conditions. Additionally, we aim to integrate the system with real ECUs in a vehicle or on a lab-grade test bench to assess performance in the presence of electromagnetic noise, voltage fluctuations, and typical signal interference found in automotive environments.

B. Adaptive Defense Against Sophisticated Adversaries

Initial findings show that simple iterative attackers cannot reliably maintain synchronization during bitrate hopping. However, more advanced adversaries could attempt partial synchronization or pattern inference. Future work will evaluate the robustness of bitrate hopping under adversarial probing and introduce a model of a 'smart attacker' that dynamically adapts its bitrate guessing strategy. We also plan to explore the use of tunable system parameters, such as hop interval granularity, entropy of the pseudo-random generator, and response delay thresholds, to defend against such adversaries. In addition, we will investigate dynamic hopping strategies triggered by anomaly detection mechanisms to increase unpredictability under suspected attack conditions.

C. Delay Spike Analysis and Mitigation

Although bitrate hopping maintains an average communication delay below 1 millisecond, transient delay spikes and message loss occur during synchronization events. Future work will conduct a detailed analysis of hop timing, bitrate mismatch severity, and CAN error frame behavior. Strategies such as timing hops to avoid midframe transitions, introducing buffer periods, or selectively delaying safety-critical messages during hopping windows will be explored to mitigate disruptions. These extensions will inform whether bitrate hopping can evolve from a reactive limp-home safeguard into a first-class security mechanism for production automotive platforms.

IX. Conclusion

This work proposes bitrate hopping as a lightweight and proactive intrusion prevention technique for Controller Area Network (CAN) systems. Unlike traditional security solutions that rely on modifications to the CAN message payload, such as message authentication, bitrate hopping operates at the physical layer, periodically altering the CAN bitrate among trusted ECUs in a pseudorandom, forward-secret manner. This desynchronizes attackers, prevents message injection, and mitigates passive sniffing without modifying the CAN frame structure or introducing cryptographic overhead per message.

We implemented and evaluated both Ethernet-coordinated and CAN-only versions of bitrate hopping using Raspberry Pi-based prototypes. The results show that the message success rates remain high ($\geq$ 98%) with Ethernet-based synchronization and moderate hop intervals, while CAN-only coordination offers backward compatibility with some trade-offs in latency and reliability at faster hop rates.

Our findings indicate that bitrate hopping is a viable mechanism for improving in-vehicle cybersecurity, particularly to isolate compromised subsystems or enforce limp-home functionality under attack. Future work will focus on real-world vehicle integration, adaptive hopping strategies, enhanced synchronization, and an in-depth investigation into delay spikes to further mature the system for production use.

Authorized licensed use limited to: Notre Dame University - Louaize. Downloaded on September 21,2026 at 19:58:19 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] Miguel Leon Chavez, Carlos Hernandez Rosete, and Francisco Rodriguez Henriquez. Achieving confidentiality security service for can. In *Electronics, Communications and Computers, 2005. CONIELECOMP 2005. Proceedings. 15th International Conference on*, pages 166–170. IEEE, 2005.

[2] Kun Cheng, Yuebin Bai, Yuan Zhou, Yun Tang, David Sanan, and Yang Liu. Caneleon: Protecting can bus with frame id chameleon. *IEEE Transactions on Vehicular technology*, 69(7):7116–7130, 2020.

[3] Kyong-Tak Cho and Kang G Shin. Error handling of in-vehicle networks makes them vulnerable. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, pages 1044–1055, 2016.

[4] Tsvika Dagan and Avishai Wool. Parrot, a software-only anti-spoofing defense system for the can bus. In *14th Embedded Security in Cars*. 2016.

[5] Paulo Freitas De Araujo-Filho, Antônio J Pinheiro, Georges Kaddoum, Divanilson R Campelo, and Fabio L Soares. An efficient intrusion prevention system for can: Hindering cyber-attacks with a low-cost platform. *IEEE Access*, 9:166855–166869, 2021.

[6] Andre Groll and Christoph Ruland. Secure and authentic communication on existing in-vehicle networks. In *Intelligent Vehicles Symposium, 2009 IEEE*, pages 1093–1097. IEEE, 2009.

[7] Bogdan Groza, Pal-Stefan Murvay, Anthony Van Herrewege, and Ingrid Verbauwhede. Libra-can: A lightweight broadcast authentication protocol for controller area networks. In *CANS*, pages 185–200. Springer, 2012.

[8] Bogdan Groza and Stefan Murvay. Efficient protocols for secure broadcast in controller area networks. *IEEE Transactions on Industrial Informatics*, 9(4):2034–2042, 2013.

[9] Road vehicles Controller area network (CAN). Standard, International Organization for Standardization, Geneva, CH, December 2015.

[10] Chung-Wei Lin and Alberto Sangiovanni-Vincentelli. Cyber-security for the controller area network (can) communication protocol. In *Cyber Security (CyberSecurity), 2012 International Conference on*, pages 1–7. IEEE, 2012.

[11] Tsutomu Matsumoto, Masato Hata, Masato Tanabe, Katsunari Yoshioka, and Kazuomi Oishi. A method of preventing unauthorized data transmission in controller area network. In *2012 IEEE 75th Vehicular Technology Conference (VTC Spring)*, pages 1–5. IEEE, 2012.

[12] Charlie Miller and Chris Valasek. Adventures in automotive networks and control units. *DEF CON*, 21:260–264, 2013.

[13] Charlie Miller and Chris Valasek. Remote exploitation of an unaltered passenger vehicle. *Black Hat USA*, 2015, 2015.

[14] Dennis K Nilsson, Ulf E Larson, and Erland Jonsson. Efficient in-vehicle delayed data authentication based on compound message authentication codes. In *Vehicular Technology Conference, 2008. VTC 2008-Fall. IEEE 68th*, pages 1–5. IEEE, 2008.

[15] Hendrik Schweppe, Yves Roudier, Benjamin Weyl, Ludovic Apvrille, and Dirk Scheuermann. Car2x communication: securing the last meter-a cost-effective approach for ensuring trust in car2x applications using in-vehicle symmetric cryptography. In *Vehicular Technology Conference (VTC Fall), 2011 IEEE*, pages 1–5. IEEE, 2011.

[16] Christopher Szilagyi and Philip Koopman. Flexible multicast authentication for time-triggered embedded control network applications. In *Dependable Systems & Networks, 2009. DSN'09. IEEE/IFIP International Conference on*, pages 165–174. IEEE, 2009.

[17] Mohammad Arman Ullah, Sheikh Ghafoor, Mike Rogers, Stacy Prowell, and Ambareen Siraj. Seccan: A practical secure control area network for automobiles. In *Annual Computer Security Applications Conference*. ACM, 2018.

[18] Anthony Van Herrewege, Dave Singelee, and Ingrid Verbauwhede. Canauth-a simple, backward compatible broadcast authentication protocol for can bus. In *ECRYPT Workshop on Lightweight Cryptography*, volume 2011, 2011.

[19] Miki E Verma, Michael D Iannacone, Robert A Bridges, Samuel C Hollifield, Bill Kay, and Frank L Combs. Road: The real ornl automotive dynamometer controller area network intrusion detection dataset (with a comprehensive can ids dataset survey & guide). *arXiv preprint arXiv:2012.14600*, 2020.

[20] Samuel Woo, Hyo Jin Jo, and Dong Hoon Lee. A practical wireless attack on the connected car and security protocol for in-vehicle can. *IEEE Transactions on Intelligent Transportation Systems*, 16(2):993–1006, 2015.

ACKNOWLEDGMENT

Notice: This manuscript has been authored by UT-Battelle, LLC, under contract DE-AC05-00OR22725 with the US Department of Energy (DOE). The US government retains and the publisher, by accepting the article for publication, acknowledges that the US government retains a nonexclusive, paid-up, irrevocable, worldwide license to publish or reproduce the published form of this manuscript, or allow others to do so, for US government purposes. DOE will provide public access to these results of federally sponsored research in accordance with the DOE Public Access Plan (https://www.energy.gov/doe-public-access-plan).

Additionally, this work was partially funded by the NCAE-C Cyber Curriculum and Research 2020 Program NCAE-C-003-2020. Grant Number H98230-20-1-0298.

Authorized licensed use limited to: Notre Dame University - Louaize. Downloaded on September 21,2026 at 19:58:19 UTC from IEEE Xplore. Restrictions apply.